AI for Anomaly Detection

Lecture 4: Unsupervised Anomaly Detection





Lilach Goren Huber
Institute for Data Science (IDS)

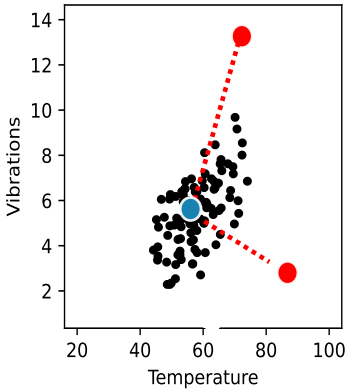
zh aw School of Engineering

WPI AnoDet

#	Date	Topic
1	15.09	Introducation to anomaly detection (AD)
2	22.09	Supervised ML for AD
3	29.09	Unsupervied ML for AD I
4	06.10	Unsupervised ML for AD II
5	13.10	Residual models: reconstruction ML methods
6	20.10	Residual models: reconstruction neural networks
7	27.10	Residual models: regression models
8	03.11	Initial assignment presentations – obligatory presence in person
9	10.11	Residual models: advanced regression neural networks (CNNs, TCNs)
10	17.11	Domain shifts and data scarcity: transfer learning
11	24.11	Contamination and mislabeling
12	01.12	Quantifying uncertainty, physics informed ML
13	08.12	Recent advances in AD research
14	**15.12**	**Final assignment presentations – obligatory presence in person**

Unsupervised AD methods

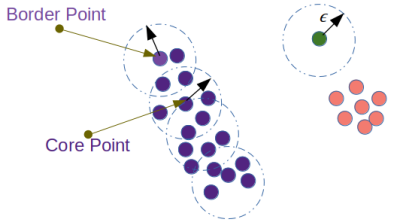
In many real-world systems we have no labels at all → detect anomalies with unsupervised methods.

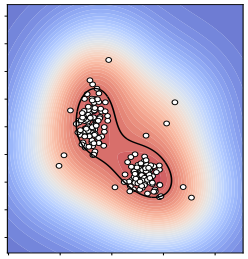


What distinguishes anomalies from the rest?

Observe the feature space:

- Higher distance → Hotelling T^2
- Lower density → DBSCAN
- Easier to separate from the others → OCSVM





Different methods rely on different criteria.

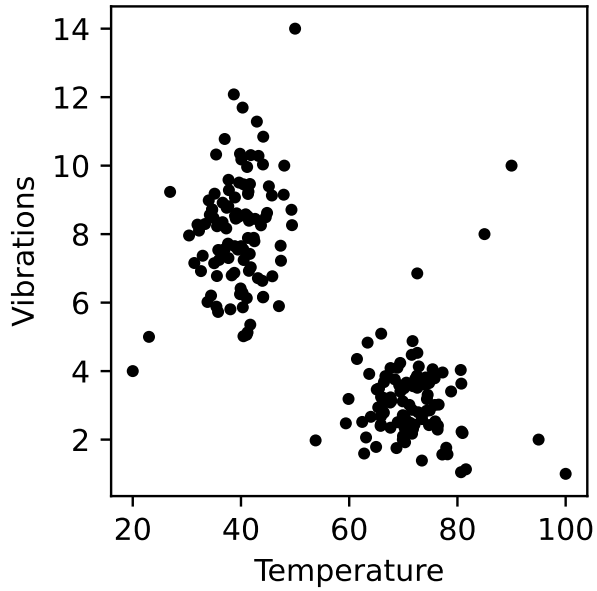
Unsupervised AD methods

- Unsupervised AD: when no labels are available during training.
- Most methods suffer from the "curse of dimensionality".
- Hyperparameters: heuristics, domain knowledge, some systematic approaches (knee in anomaly scores).
- Evaluation: either with some labeled validation data or apostriori with an expert.
- Note: **Ensemble** methods enhance robustness, and help quantify parameter sensitivity.

Unsupervised AD methods

In many real-world systems we have no labels at all ➔ detect anomalies with unsupervised methods.

- Clustering (DBSCAN example)
- One-Class SVM
- Local outlier factor (LOF)
- Isolation forest (IF)



- Higher distance → Hotelling T^2
- Lower density → DBSCAN, **LOF**
- Easier to separate from the others → OCSVM, **IF**

Today

- More unsupervised AD algorithms
  - Local Outlier Factor
  - Isolation forest
- Exercise: Unsupervised AD in Credit Card Transactions
  - Apply IF and LOF
- Unsupervised dimension reduction
  - PCA
  - t-SNE
  - UMAP
- Example: dimension reduction on FMNIST

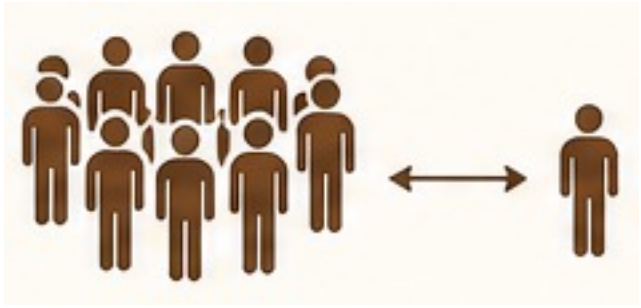
Unsupervised AD with Local Outlier Factor (LOF)

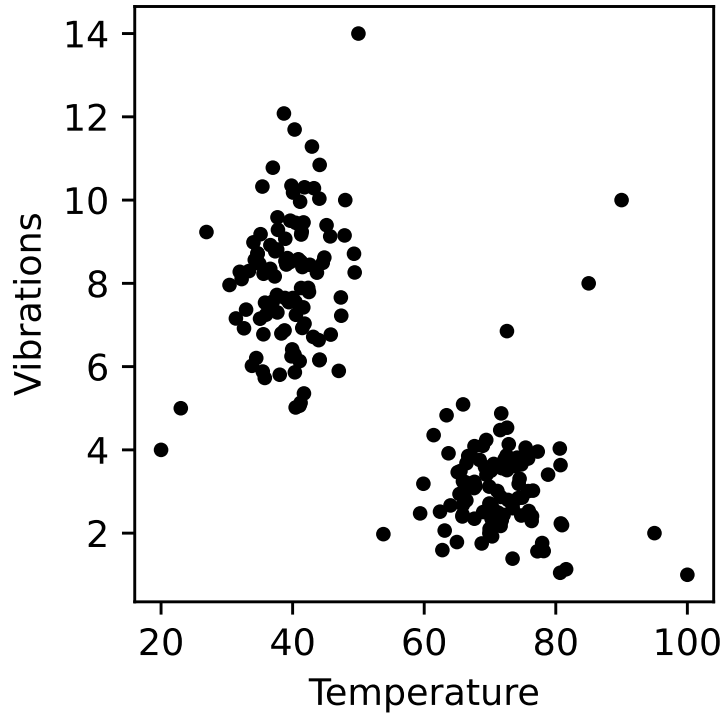
Imagine everyone is standing in a crowd. Each person asks:

"How crowded am I compared with my friends?"

- If you and your friends are equally crowded, nothing is unusual.
- If your friends are standing close together but you are standing much farther away, you look suspicious.

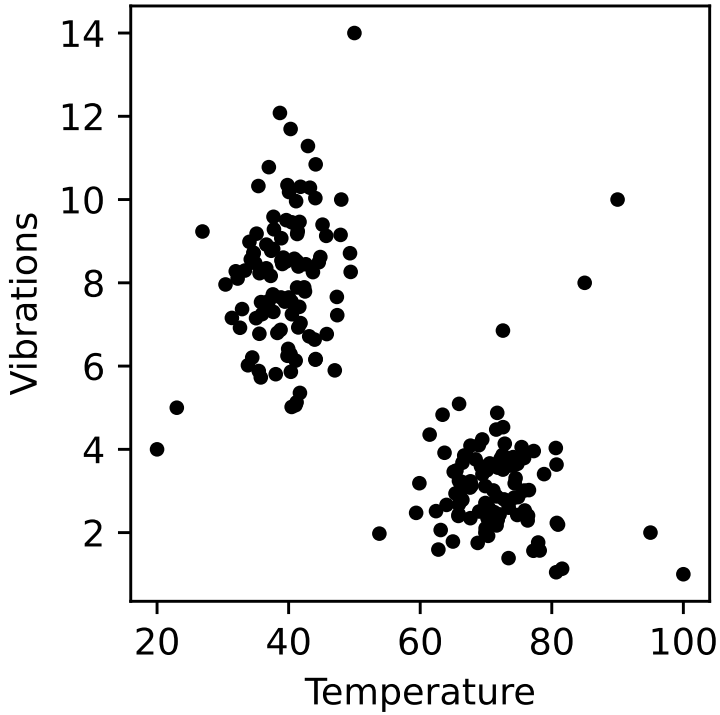
→ LOF measures exactly this relative crowding.





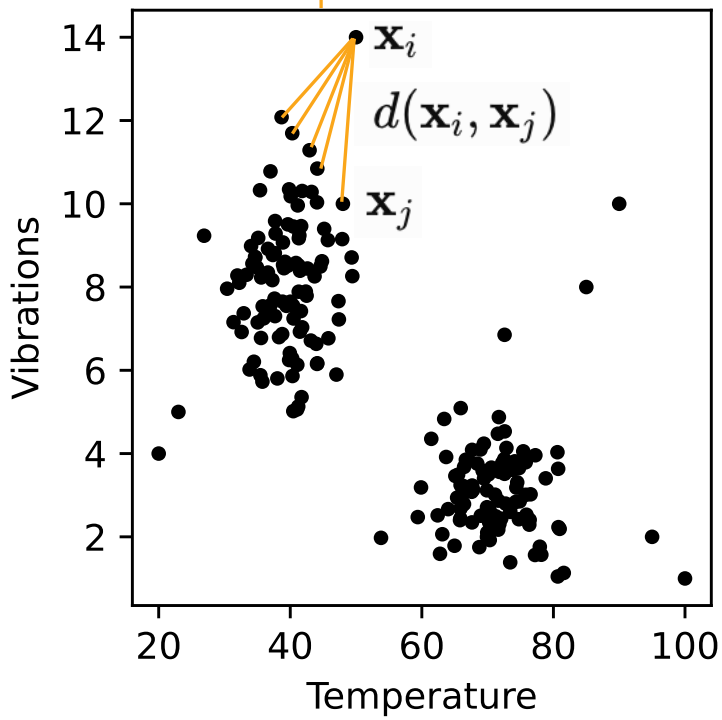
Unsupervised AD with Local Outlier Factor (LOF)

1. For each point find k-nearest neibours (k-nns).
2. Estimate their local density.
3. Identify points that have a much lower density than their neighbours.



1. For each point find k-nearest neibours (k-nns).
   - For each data point, measure its distance to all other points
   - Select the k closest points
   - These are the k-nns of this data point.

Example with k=5



$$\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id}), \qquad \mathbf{x}_j = (x_{j1}, x_{j2}, \ldots, x_{jd}),$$

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^{d} (x_{im} - x_{jm})^2}.$$

2. Estimate the local density

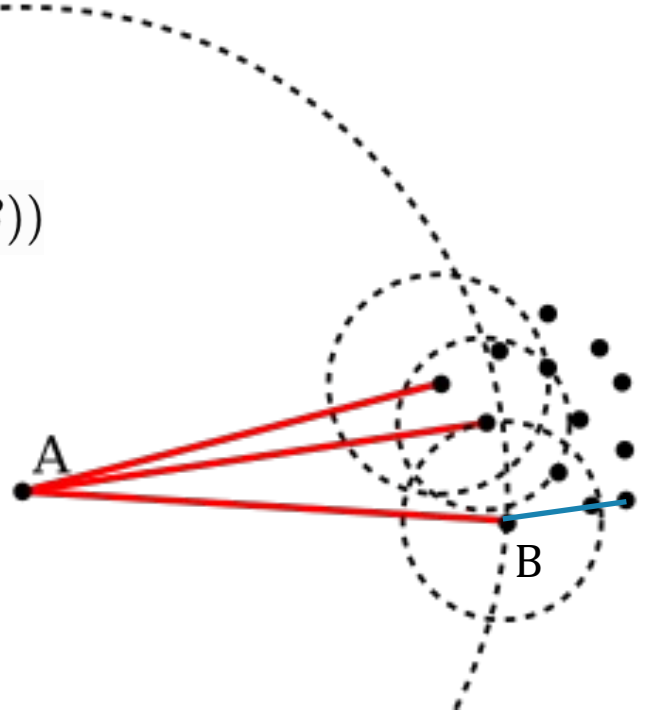
a. Calculate the reachability distance of point A and its nns.
b. Define for A its local reachability density (LRD).
c. Define for A its Local Outlier Factor (LOF).

a. for a point **A** and one of its **k**-nns **B**

$$\text{reach-dist}_k(A, B) = \max(d(A, B),\ k\text{-distance}(B))$$

If **A** lies very close to **B**, we still use at least the radius of **B**'s neighbourhood. This prevents extremely high density estimates.

distance from **B** to its **k**-th nearest neighbour.

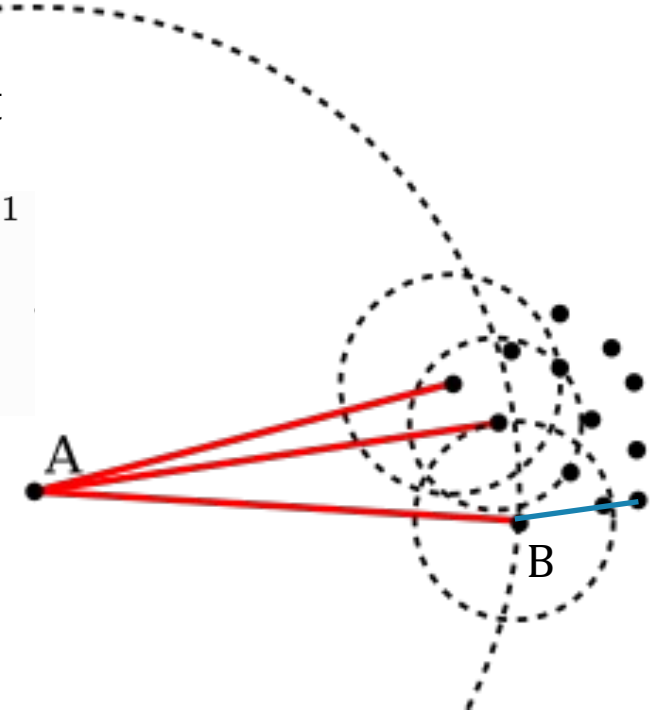


2. Estimate the local density

a. Calculate the reachability distance of point A and its nns.
b. Define for A its local reachability density (LRD).
c. Define for A its Local Outlier Factor (LOF).

b. The LRD of A is the inverse of its mean reach-dist

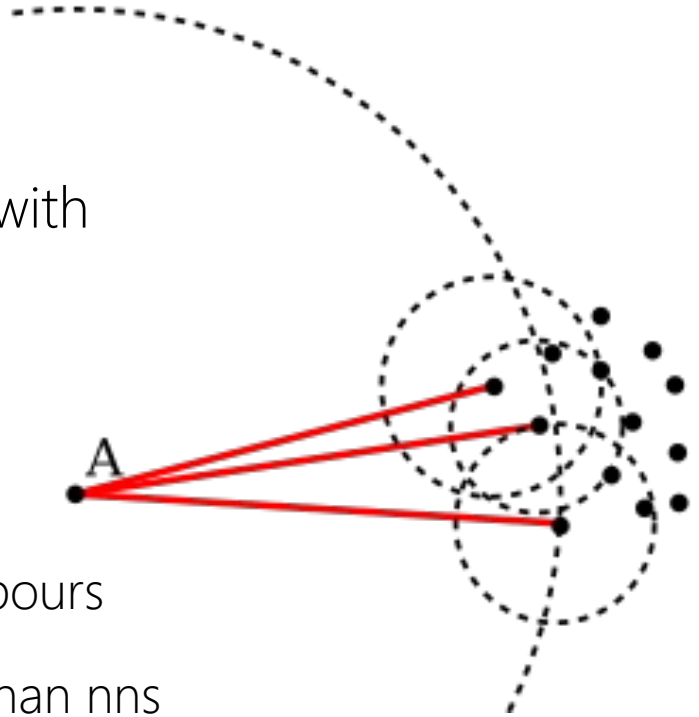
$$\mathrm{LRD}_k(A) = \left(\frac{1}{k} \sum_{B \in N_k(A)} \text{reach-dist}_k(A, B) \right)^{-1}$$



2. Estimate the local density

a. Calculate the reachability distance of point A and its nns.
b. Define for A its local reachability density (LRD).
c. Define for A its Local Outlier Factor (LOF).

c. The LOF score of point A is its LRD compared with its neighbours



$$LOF_k(A) = \frac{\sum_{B \in N_k(A)} \frac{LRD_k(B)}{LRD_k(A)}}{|N_k(A)|}$$

~ 1 Similar to neighbours

< 1 Higher density than nns

> 1 Lower density than nns (outlier)

Unsupervised AD with Local Outlier Factor (LOF)

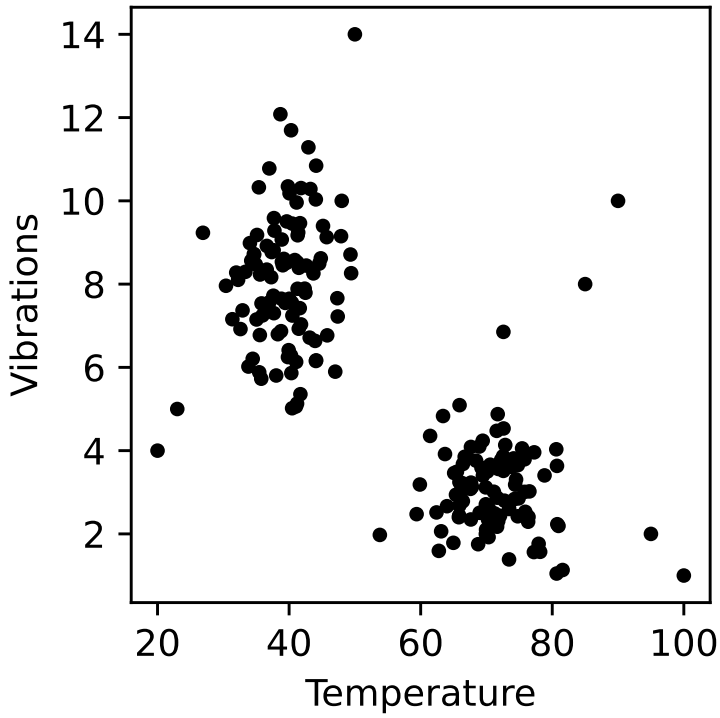
```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination="auto"
)

labels = lof.fit_predict(X)
scores = -lof.negative_outlier_factor_
```

- N_neighbors: hyperparameter.
- Contamination: used to choose the decision threshold.
  - «auto» : LOF>1.5.
  - Contamination = 0.05 labels the top 5% most anomalous points regardless of their LOF values.
- Novelty=false: fit & detect anomalies on the same data.

  =True: fit on normal data, detect on unseen data.

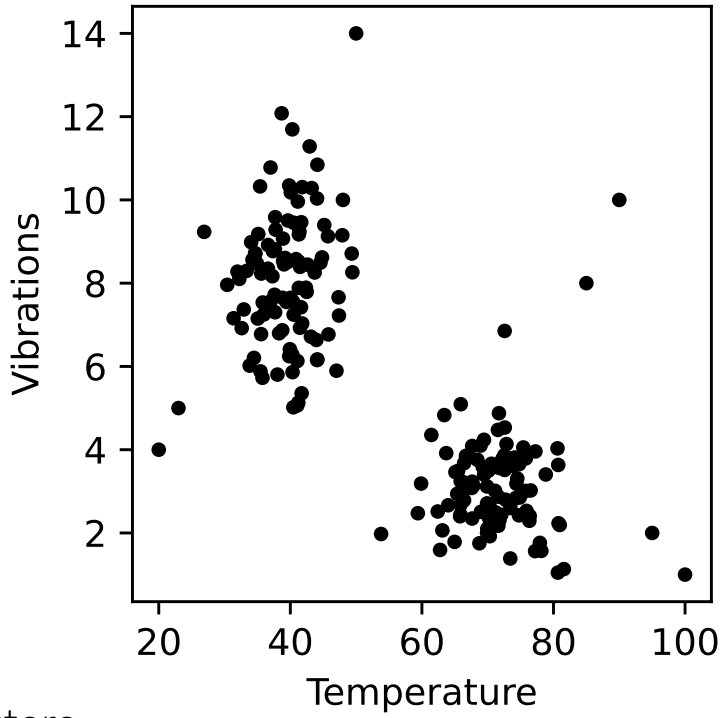


Unsupervised AD with Local Outlier Factor (LOF)

```python
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination="auto"
)

labels = lof.fit_predict(X)
scores = -lof.negative_outlier_factor_
```

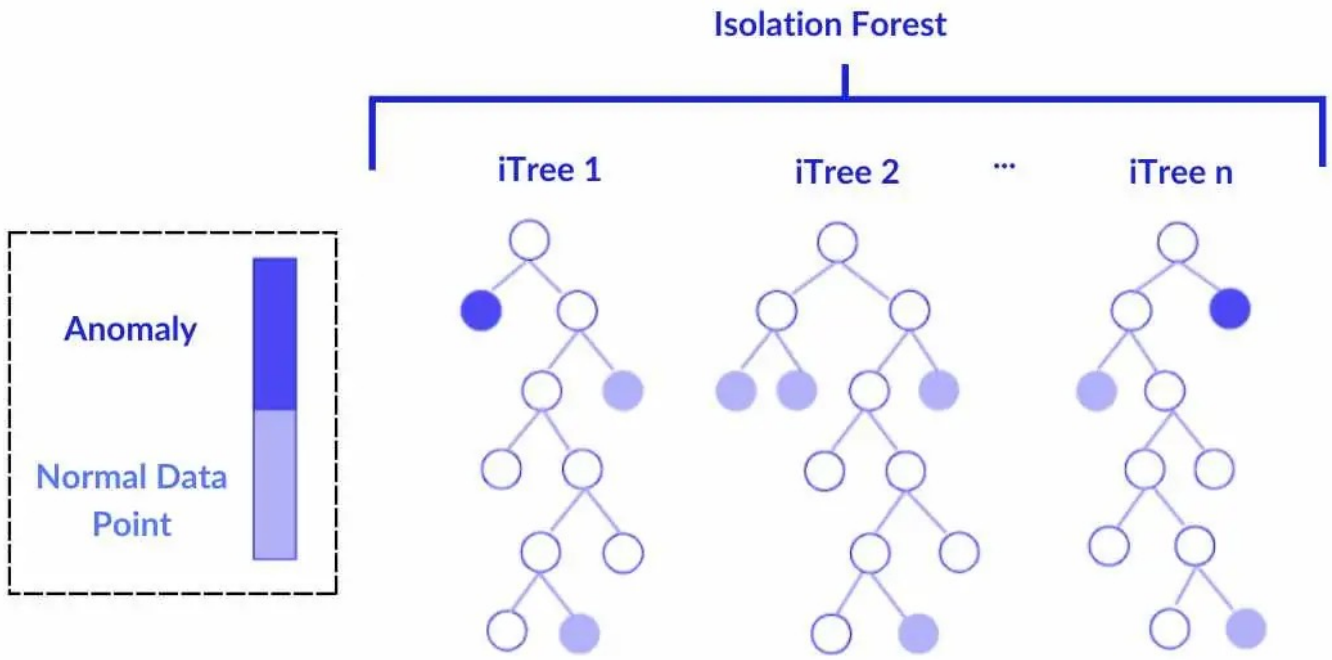


Choosing N_neighbors:

- Controls the size of the local neighbourhood.
- Small k: sensitive to local anomalies and noise.
- Large k: more stable, but less sensitive to small anomaly clusters.
- Typical values: 10–50 (default: 20).
- If labels are available, select k by cross-validation.
- Otherwise, start with k=20 or $\sim\sqrt{N}$ and assess if the detected anomalies are meaningful.

WPI_AnoDet_04a_AD_unsupervised_LOF_example_machine_data.

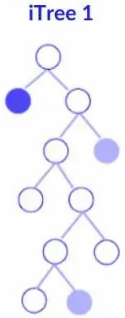
Unsupervised AD with Isolation Forest (IF)



Isolation tree

Liu, Fei Tony; Ting, Kai Ming; Zhou, Zhi-Hua (December 2008). "Isolation Forest".

https://www.datacamp.com/tutorial/isolation-forest

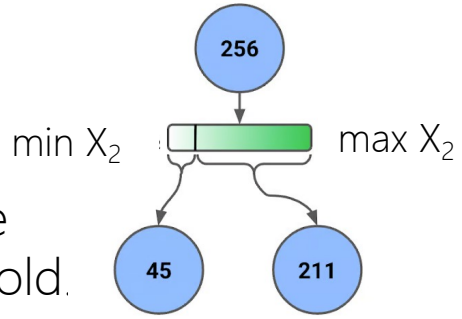


Isolation tree

The idea: anomalies are easier to isolate than normal points.

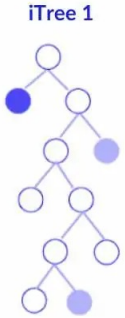
How to build one tree?

- Randomly **select** randomly one **feature** (e.g X_1 or X_2)
- Randomly **choose a split value** within the range of that feature
- **Split** the data into two groups according to the chosen threshold.
- **Repeat** the same procedure recursively for each resulting node.



.

https://www.datacamp.com/tutorial/isolation-forest

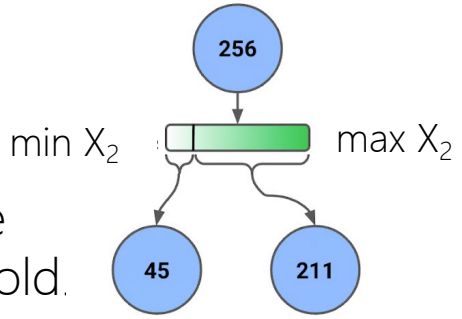


Isolation tree

The idea: anomalies are easier to isolate than normal points.

How to build one tree?

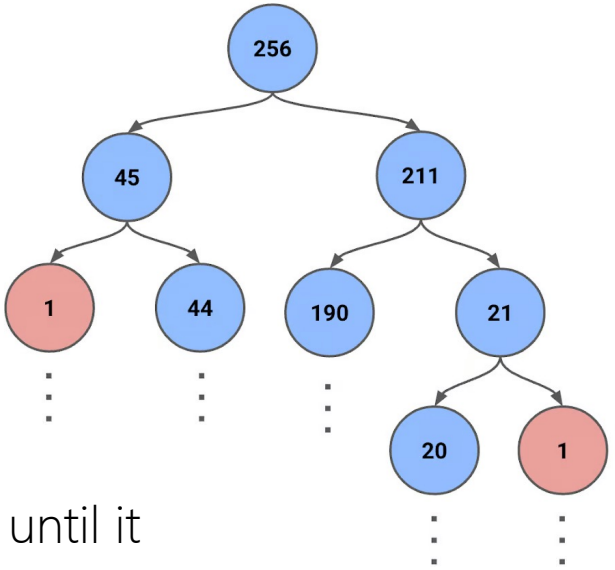
- Randomly **select** randomly one **feature** (e.g X_1 or X_2)
- Randomly **choose a split value** within the range of that feature
- **Split** the data into two groups according to the chosen threshold.
- **Repeat** the same procedure recursively for each resulting node.



.

Stop when

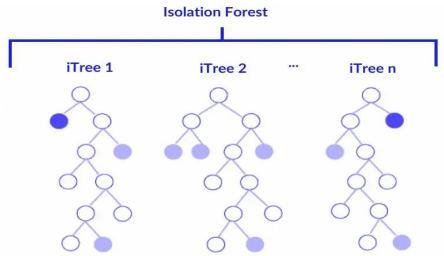
- The node contains only one instance
- Or a maximum tree depth is reached



→ For every data point $\mathbf{x}$ count how **many splits are needed** until it becomes isolated → its path length $\boldsymbol{h(x)}$

https://www.datacamp.com/tutorial/isolation-forest

Isolation forest



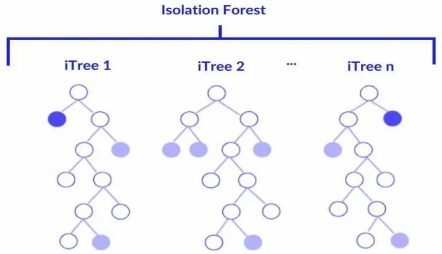
An ensemble of trees

- Build many isolation trees, each using different random splits.
- For sample x tree $\mathbf{i}$ has a path length $h_i(x)$
- Average the path lengths of each sample x over all trees →$E[h(x)]$

.

https://www.datacamp.com/tutorial/isolation-forest

Isolation forest



An ensemble of trees

- Build many isolation trees, each using different random splits.
- For sample x tree $\mathbf{i}$ has a path length $h_i(x)$
- Average the path lengths of each sample x over all trees →$E[h(x)]$

Interpretation

- Short average path → isolated quickly → likely anomaly
- Long average path → difficult to isolate → likely normal

.

→ Anomaly score $s(x,n) = 2^{-\frac{E[h(x)]}{c(n)}}$

Normalization based on sample size n

≈ 0 normal

≈ 0.5 borderline

≈ 1 very anomalous

Liu, Fei Tony; Ting, Kai Ming; Zhou, Zhi-Hua (December 2008). "Isolation Forest".

https://www.datacamp.com/tutorial/isolation-forest

Unsupervised AD with Isolation Forest (IF)

```python
from sklearn.ensemble import IsolationForest

model = IsolationForest(
    n_estimators=100,
    max_samples='auto',
    contamination='auto',
    max_features=1.0,
    random_state=42
)

model.fit(X_train)

scores = -model.score_samples(X_test)   # larger = more anomalous
labels = model.predict(X_test)          # -1 = anomaly, 1 = normal
```

score_samples(x) = $-s(x, n)$:

- normal point →0
- anomaly → -1

decision_function(x) = score_samples(x)+0.5

decision_function(x)<0 → anomaly

Hyperparameter	Default	Meaning	Typical values
n_estimators	100	Number of isolation trees	100–300
max_samples	"auto"	Number of training samples used to build each tree. Auto = max(256,$n_{samples}$).	"auto"
contamination	"auto"	Expected fraction of anomalies (used only for the decision threshold)	"auto" or known fraction (e.g. 0.01)
max_features	1.0	Fraction of features considered when building each tree	0.5–1.0
random_state	None	Random seed for reproducibility	any integer

WPI_AnoDet_04b_AD_unsupervised_isolation_forest_example_machine_data

Your exercise: Unsupervised AD in Credit Card Transactions

The datasets contain simulated credit card transactions from one customer. The customer has one or two normal purchasing patterns, depending on the dataset. A small number of transactions are anomalous and do not match the usual behavior.

- Inputs:
  - hour: Hour of the day at which the transaction occurred, between 0 and 24.
  - amount: Transaction amount in CHF.

- Target:
  - Normal or anomaly.

- Task: detect abnormal transactions with
  - Isolation Forest
  - Local Outlier Factor (LOF)

RECENT TRANSACTIONS

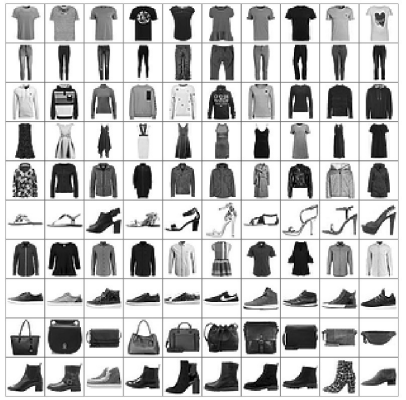
DATE	AMOUNT	MERCHANT	LOCATION	SCORE	FLAG
May 26, 10:15	$23.50	Coffee House	New York, USA	-0.12	✓
May 26, 12:47	$57.90	SuperMart	New York, USA	-0.15	✓
May 26, 14:05	$89.99	Amazon	New York, USA	-0.18	✓
May 26, 16:20	$1,250.00	ElectroWorld	Miami, USA	-0.72	⚠
May 26, 18:33	$19.90	Pharmacy	New York, USA	-0.10	✓
May 26, 21:17	$2,340.00	Luxury Watch Co.	Paris, France	-0.89	⚠
May 27, 09:02	$25.00	Bakery	New York, USA	-0.11	✓

WPI_AnoDet_04c_exercise_AD_unsupervised_LOF_IF

Unsupervised dimension reduction for visualization

- We saw that visualizing the data helps find the best approach.
- High dimensional data → visualiziation is limited
- Pairplot gives max 2D correlations between variables

→Reduce dimensions to visualize in 2D



Approaches:

1. Principal Component Analysis (PCA)
2. t-distributed Stochastic Neighbor Embedding (t-SNE)
3. Uniform Manifold Approximation and Projection (UMAP)

Dimension reduction with Principal Component Analysis (PCA)

Goal: reduce the dimensionality from $\mathbf{d}$ variables to $\mathbf{k} < \mathbf{d}$ variables while preserving as much information as possible.

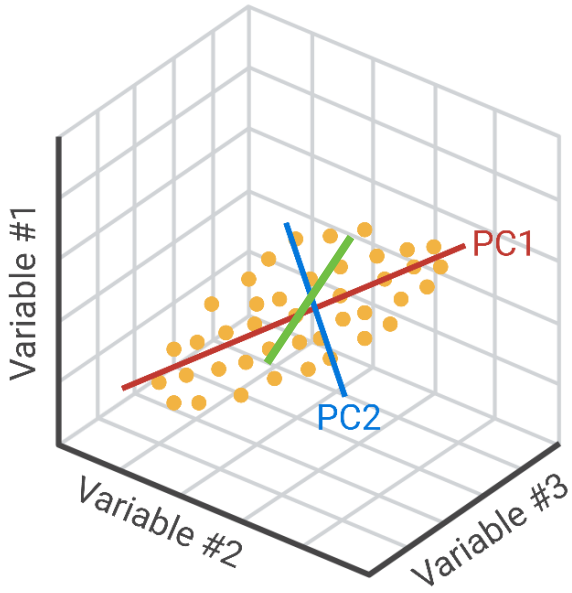
- Imagine the data as a cloud of points in 3 dimensions.
- Find the direction in which the data spreads out the most.

→This is the 1st principal component (PC1).

- Find the next direction with the largest remaining spread, perpendicular to the first.

→2nd PC (PC2).

- Continue until all PCs are found.



Dimension reduction with Principal Component Analysis (PCA)

Goal: reduce the dimensionality from $\mathbf{d}$ variables to $\mathbf{k} < \mathbf{d}$ variables while preserving as much information as possible.

- Imagine the data as a cloud of points in 3 dimensions.
- Find the direction in which the data spreads out the most.

→This is the 1st principal component (PC1).

- Find the next direction with the largest remaining spread, perpendicular to the first.

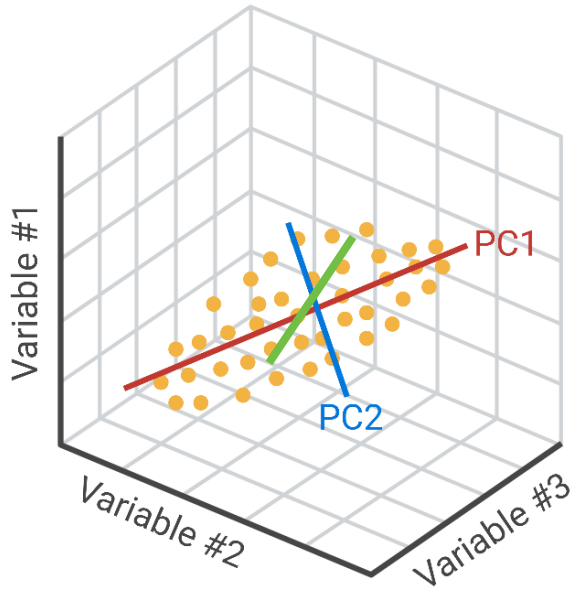
→2nd PC (PC2).

- Continue until all PCs are found.

→ Directions with large variance contain more information than directions with small variance.

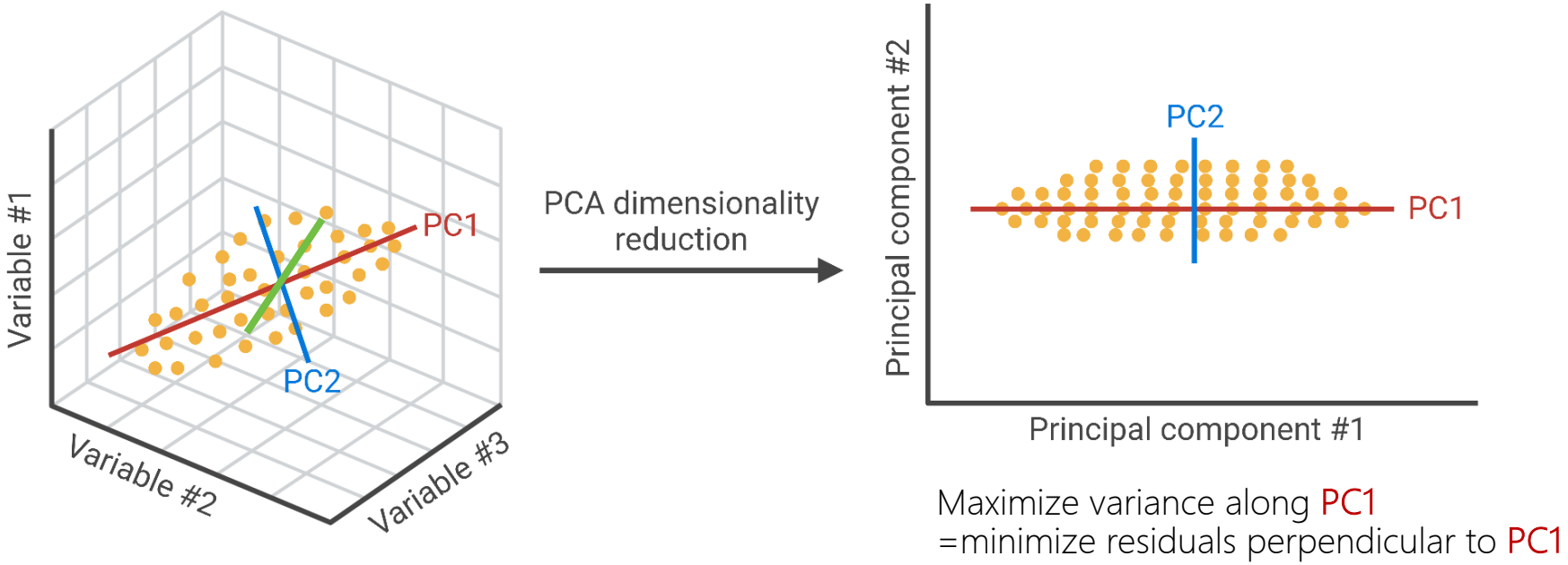
→ Remove directions with small variance

→ Reduce dimensionality while maximizing information.



PCA can be viewed as fitting a $\mathbf{d}$-dimensional ellipsoid to the data:

- fit a d-dimensional ellipsoid to the data using a linear transformation.
- Large ellipsoid axes → high variance
- Small axis → variable does not explain much → less important → drop it.



Maximize variance along PC1
=minimize residuals perpendicular to PC1

Image source: https://www.biorender.com/template/principal-component-analysis-pca-transformation

Dimension reduction with PCA

- $X^{(0)}$ is the data matrix (n samples, d features).
- Center the data: $x_{ij} = x_{ij}^{(0)} - \mu_j$
- Calculate covariance matrix $\Sigma = \frac{1}{n-1} X^T X = \frac{1}{n-1} \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}^T \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}$
- Σ_{ij} is the cov. Between feature i and j.
- Σ_{ii} is the variance of feature i.

Dimension reduction with PCA

- $X^{(0)}$ is the data matrix (n samples, d features).
- Center the data: $x_{ij} = x_{ij}^{(0)} - \mu_j$
- Calculate covariance matrix $\Sigma = \frac{1}{n-1} X^T X = \frac{1}{n-1} \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}^T \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}$
- Σ_{ij} is the cov. Between feature i and j.
- Σ_{ii} is the variance of feature i.
- Calculate the eigenvectors v_m and eigenvalues λ_m of Σ.

Independent directions along which the data varies the most

How much the data varies along this direction

- $X^{(0)}$ is the data matrix (n samples, d features).
- Center the data: $x_{ij} = x_{ij}^{(0)} - \mu_j$
- Calculate covariance matrix $\Sigma = \frac{1}{n-1} X^T X = \frac{1}{n-1} \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}^T \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}$
- Σ_{ij} is the cov. Between feature i and j.
- Σ_{ii} is the variance of feature i.
- Calculate the eigenvectors v_m and eigenvalues λ_m of Σ.

Independent directions along which the data varies the most

How much the data varies along this direction

- Sort the eigenvectors by decsending eigenvalues.
- Select the top $\mathbf{k}$ eigenvectors with the highest eigenvalues.
- Form the projection matrix $W = [v_1, v_2, \ldots, v_k]$
- The transformed data is $\mathrm{Z} = \mathrm{X}\, W$

$(n \times k)$ $(n \times d)$ $(d \times k)$

A full PCA is a rotation of the coordinate system using a $d \times d$ matrix V:

$$Z = XV \qquad \text{(all } d \times d \text{)}$$

It is not just **any** rotation, but one that **diagonalizes** the covarianve matrix:

$$\Sigma_Z = V^\top \Sigma V = \Lambda,$$

Covariance of X — Covariance of Z

Diagonal with all eigenvalues
→the projected variables are **uncorrelated**

$$\mathrm{Var}(Z_i) = \lambda_i$$

This is all correct without reducing dimensions.

PCA math: dimension reduction

$$\Sigma_Z = V^\top \Sigma V = \Lambda,$$

Full PCA: diagnalization of the covariance matrix

Covariance of $\mathbf{X}$

Covariance of $\mathbf{Z}$

Diagonal with all eigenvalues
→the projected variables are **uncorrelated**

$$\mathrm{Var}(Z_i) = \lambda_i.$$

Order the eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$

Select the eigenvectors with the highest eigenvalues $W = [v_1, \ldots, v_k]$

Transform $\mathbf{X}$ with the new matrix W

$$\mathbf{Z} = \mathbf{X}\,W$$

$(n \times k)$ $(n \times d)$ $(d \times k)$

W diagonalizes the covariance matrix keeping only the k highest eigenvalues

$$\Sigma_Z = W^\top \Sigma W = \Lambda_k$$

PCA implementation

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_pixels)
```

$= \texttt{pca.fit(X_pixels)} + \texttt{X_pca = pca.transform(X_pixels)}$

$Z = X\,W$

- Center the data: $x_{ij} = x_{ij}^{(0)} - \mu_j$
- Find the principal directions

Perform the transformation

Dimension reduction with t-distributed Stochastic Neighbor Embedding (t-SNE)

Goal: visualize high-dimensional data $\mathbf{x}_1, \ldots, \mathbf{x}_N$ in 2D or 3D by representing it as $\mathbf{z}_1, \ldots, \mathbf{z}_N$ placing similar points close together and dissimilar points far apart.

Goal: visualize high-dimensional data $\mathbf{x}_1, \ldots, \mathbf{x}_N$ in 2D or 3D by representing it as $\mathbf{z}_1, \ldots, \mathbf{z}_N$ placing similar points close together and dissimilar points far apart.

Step 1: Measure neighbourhood probabilities in the original space

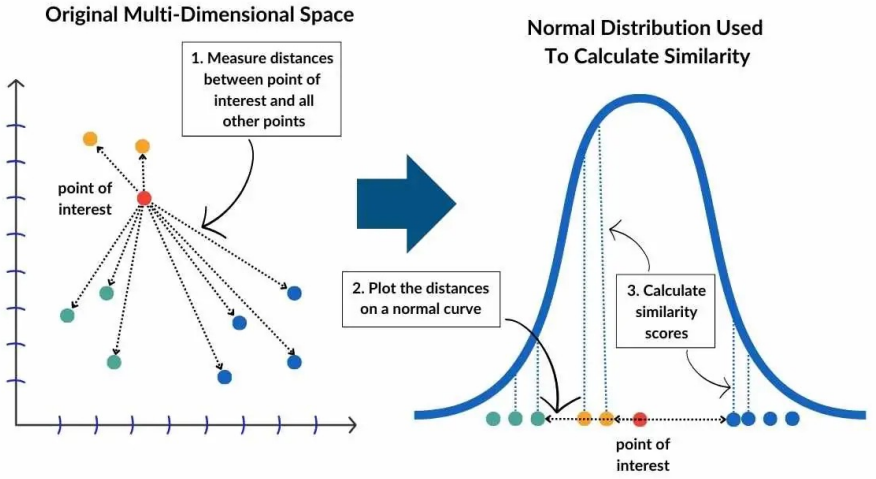
$$p_{j|i} = \frac{\exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma_i^2}\right)}{\sum_{k \neq i} \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_k\|^2}{2\sigma_i^2}\right)} \xrightarrow{\text{symmetrize}} p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}.$$

$p_{j|i}$: probability that x_j is a neighbor of x_i (neighborhood defined by σ_i)

For every point $\mathbf{i}$:

- Nearby points receive a **high probability**.
- Distant points receive a **low probability**.

➔ each point gets a similarity score to each other point.



Step 2: Create an initial 2D embedding $\mathbf{z}_1, \ldots, \mathbf{z}_N$

- Place all points randomly
- Define similarities in 2D

$$q_{ij} := \frac{\left(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|\mathbf{z}_k - \mathbf{z}_l\|^2\right)^{-1}}$$

In embedding space let probabilities drop slowly with distance (quadratic and not exponential like a Gaussian distribution):

$$f(t) = \frac{1}{\pi(1 + t^2)}$$

→ Moderately dissimilar points are far apart in embedding space

→ Stronger repulsion between unrelated points → clusters are better separated

Step 3: Improve the 2D embedding by matching the two distributions

- The algorithm repeatedly moves the points in 2D.
- If two points are neighbours in the original space (high p_{ij}) but are far apart → pull them together.
- If two points are close in the map but not originally (low p_{ij}) → push them apart.

- Continue until the neighbourhood relationships in 2D resemble those in the original space.

Step 3: Improve the 2D embedding by matching the two distributions

- The algorithm repeatedly moves the points in 2D.
- If two points are neighbours in the original space (high p_{ij}) but are far apart → pull them together.
- If two points are close in the map but not originally (low p_{ij}) → push them apart.

- Continue until the neighbourhood relationships in 2D resemble those in the original space.

- Mathematically done by minimizing the Kullback-Leibler divergence between the
  - Pairwise similarity distribution in the original high-dimensional space $P = \{p_{ij}\}$
  - Pairwise similarity distribution in the low-dimensional embedding space $Q = \{q_{ij}\}$

$$KL(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

  - High p_{ij} , low $q_{ij.}$ → push i,j towards each other to increase $q_{ij.}$
  - Low p_{ij} , high q_{ij} → small loss contribution

https://www.youtube.com/watch?v=U-s8q6HshZw

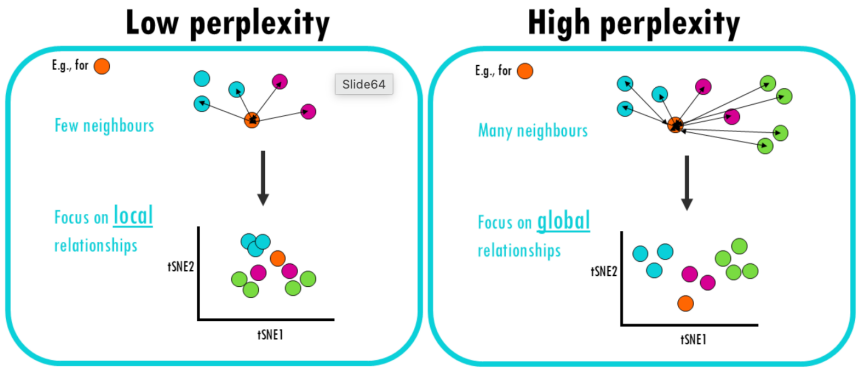
t-SNE implementation

```python
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# X: (n_samples, n_features)

tsne = TSNE(
    n_components=2,
    perplexity=30,
    learning_rate="auto",
    max_iter=1000,
    init="pca",
    random_state=42
)

X_2d = tsne.fit_transform(X)
```

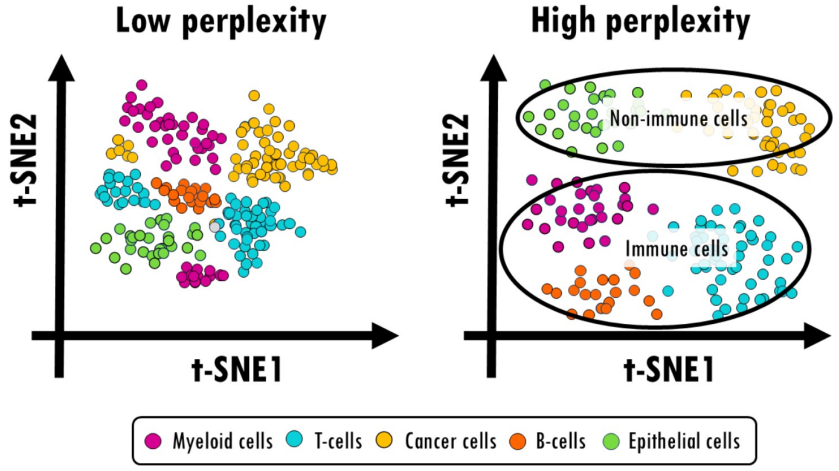


effective number of neighbours each point considers
Usually between 5 and 50.

Start with the 2D PCA projection as initial embedding

*The algorithm adjusts σ_i for each point to obtain the specified perplexity defined as

$$2^{-\sum_{j \neq i} p_{j|i} \log_2 p_{j|i}}$$



Dimension reduction with Uniform Manifold Approximation & Projection (UMAP)

Goal: reduce the dimensionality from $\mathbf{d}$ variables to 2D or 3D while trying to preserve the data's neighbourhood structure.

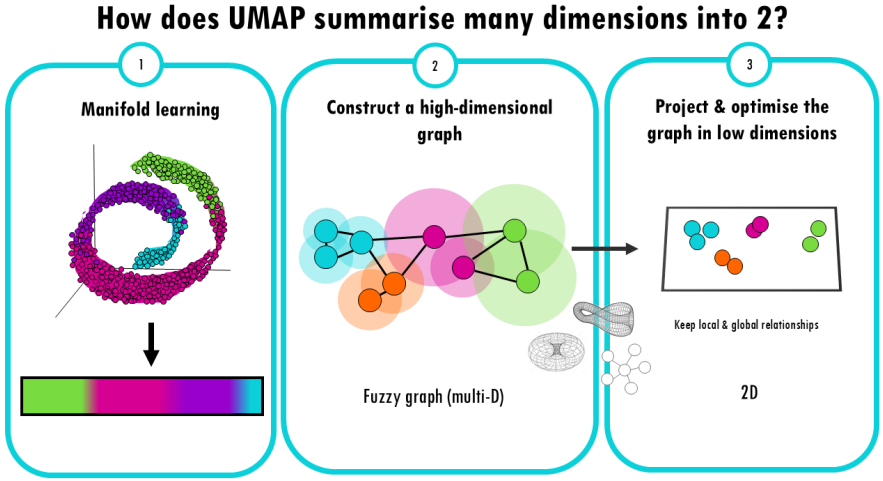
- Find the k-nns of each point in the original space
- Construct a weighted graph: nearby points are connected strongly.
- Place the points in 2D
- Move the points until the neighbour relationships in 2D resemble those of the original graph

Dimension reduction with Uniform Manifold Approximation & Projection (UMAP)

Goal: reduce the dimensionality from $\mathbf{d}$ variables to 2D or 3D while trying to preserve the data's neighbourhood structure.

- Find the k-nns of each point in the original space
- Construct a weighted graph: nearby points are connected strongly
- Place the points in 2D
- Move the points until the neighbour relationships in 2D resemble those of the original graph

- like t-SNE, UMAP **optimizes** the layout in 2D.
- Uses a technique from topology (the study of shapes) to optimize the **graph** in a lower-dimensional space.

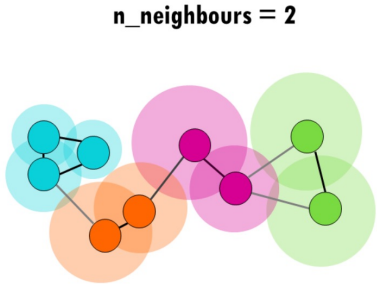


The idea: keep the local neighbourhood relationships, while also trying to preserve the global relationships in the data.

https://biostatsquid.com/umap-simply-explained/

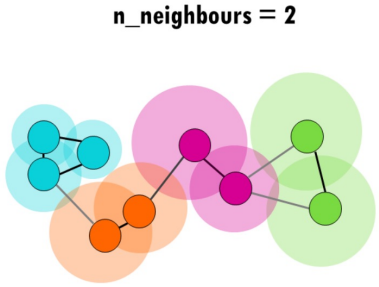
Step 1: for every original data point $\mathbf{x_i}$ find its k nearest neighbours.

- k is a hyperparameter.
- $N_k(i)$ is the set of its nearest neighbours



Step 1: for every original data point $\mathbf{x_i}$ find its k nearest neighbours.

- k is a hyperparameter.
- $N_k(i)$ is the set of its nearest neighbours



Step 2: convert distances into connection strengths.

- For point i define the connection strength to point j as

$$p_{j|i} = 0$$

if $j \notin N_k(i)$

$$p_{j|i} = \exp\left(-\frac{\max(0, d(\mathbf{x}_i, \mathbf{x}_j) - \rho_i)}{\sigma_i}\right)$$

distance between the points

controls the local distance scale around point i. Different regions may have different densities

~distance from point i to its closest neighbour

$$\rho_i = \min_{j \in N_k(i)} d(i, j)$$

Step 3: make the graph symmetric $p_{ij} = p_{j|i} + p_{i|j} - p_{j|i}p_{i|j}$

- Analogous to the probability of the union of two events

$$P(A \cup B) = P(A) + P(B) - P(A)P(B)$$

Step 4: define the similarity in 2D space

$$q_{ij} = \frac{1}{1 + a \left\| \mathbf{z}_i - \mathbf{z}_j \right\|^{2b}}$$

behaves somewhat like the Student-t kernel:
- nearby points have $q_{ij} \approx 1$
- distant points have $q_{ij} \approx 0$

Hyperparameters - chosen based mainly on **min_dist**.

Step 5: match the two graphs $q_{ij} \approx p_{ij}$

- By minimizing a binary cross-entropy-like objective:

$$\mathcal{L} = -\sum_{i \neq j} [p_{ij} \log q_{ij} + (1 - p_{ij}) \log(1 - q_{ij})]$$

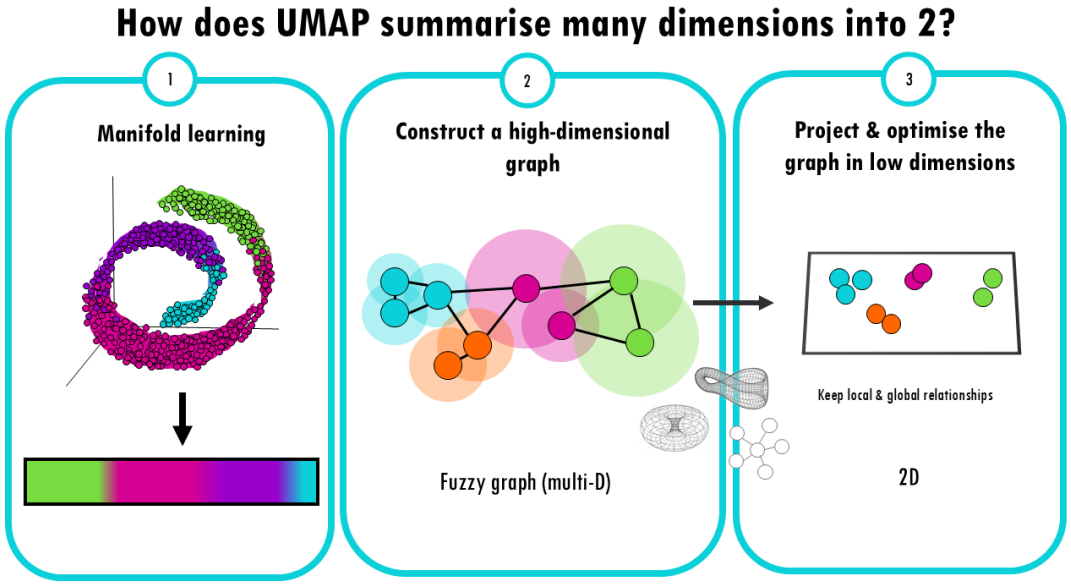
attractive repulsive

Step 5: match the two graphs $q_{ij} \approx p_{ij}$

- By minimizing a binary cross-entropy-like objective:

$$\mathcal{L} = -\sum_{i \neq j} [p_{ij} \log q_{ij} + (1 - p_{ij}) \log(1 - q_{ij})]$$

attractive repulsive



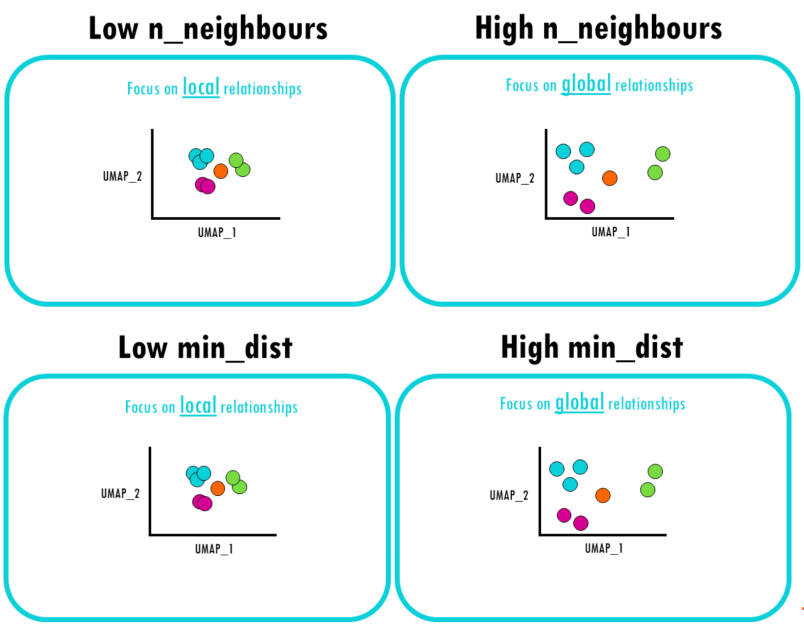
Dimension reduction with UMAP

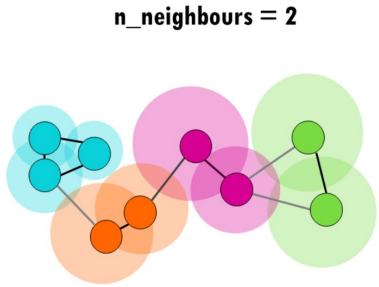
```
model = umap.UMAP(
    n_components=2,
    n_neighbors=15,
    min_dist=0.1,
    metric="euclidean",
    random_state=42,
)
```

Larger neighbourhood →the algorithm is forced to preserve relationships over a broader region

Controls how tightly embedded points can be grouped

Can choose different metrics





Unsupervised dimension reduction for visualization

- PCA
- t-SNE
- UMAP

- Created by Zalando
- 10 article classes
- 60000 training images
- 10000 test images
- Typically used for classification benchmarking.
- We will use it for image anomaly detection benchmarking.

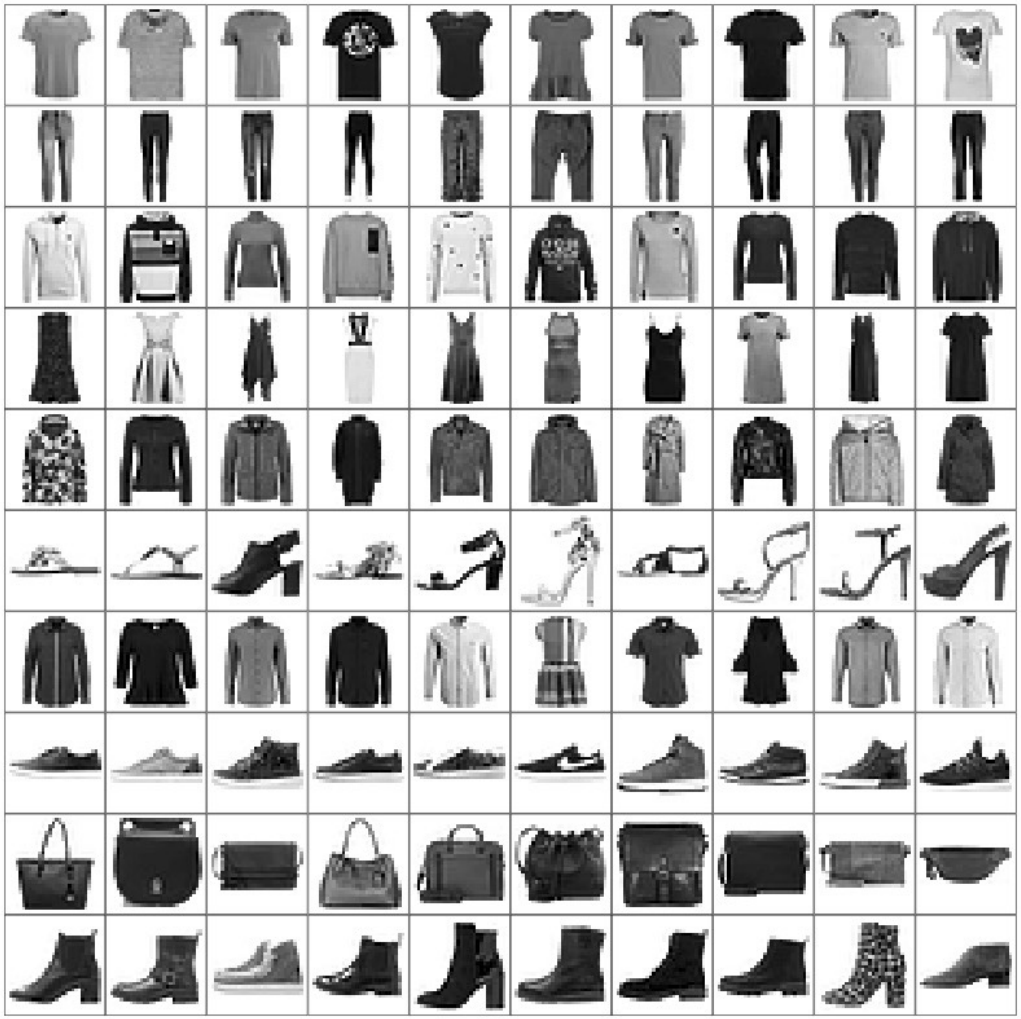


Unsupervised dimension reduction for visualization

- PCA
- t-SNE
- UMAP

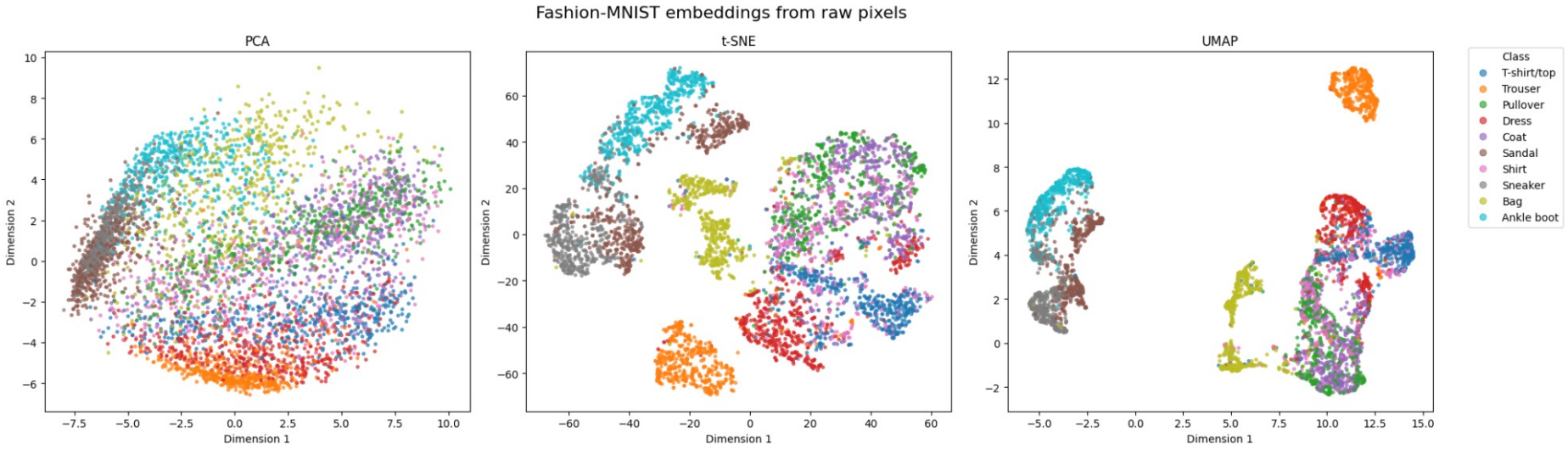
Questions

- What is the original space?
- What dimension?
- What would you expect from a good 2D embedding?
- Which method do you think will work best? why?



WPI_AnoDet_04d_FMNIST_dimension_reduction_visualization

Results

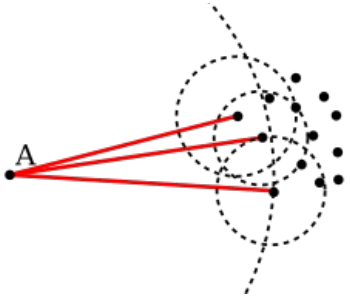


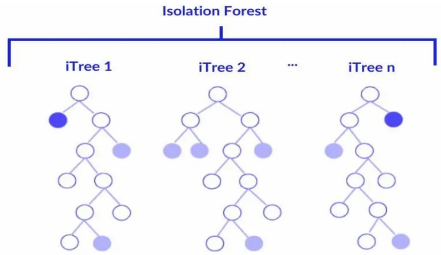
Cluster structure preserved

Inter-cluster distances are enhanced

Summary

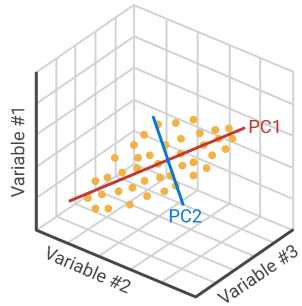
- Unsupervised AD algorithms
  - Local Outlier Factor
  - Isolation forest

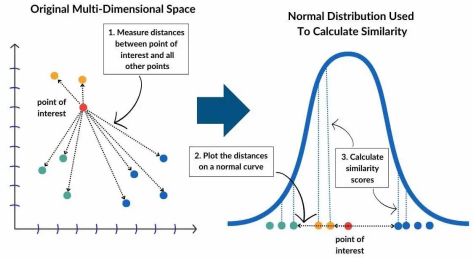




  - Exercise: Unsupervised AD in Credit Card Transactions
    - Apply IF and LOF

- Unsupervised dimension reduction
  - PCA
  - t-SNE
  - UMAP





  - Example: dimension reduction on FMNIST

